S6 — Hardware Alarm Sense & Reverse-Power-Flow Adaptive Threshold

OpenAMI / MeshEMS — Continuous Leakage Detection (NESL)

Allahaddje Bonheur — IoT Fellow, 10Power (in collaboration with NESL)

2026-07-27

1 Objective

Sprint **S6** closes the loop between the MD0630 and the EMS in two directions:

- **(A) Hardware alarm sense** — the module trips on its **own** internal relay when leakage crosses a threshold (the life-safety path, independent of firmware). S6-A lets the EMS **observe** those alarm output lines on ESP32 GPIO so a trip is logged and published — the firmware never drives the trip.
- **(B) Reverse-power-flow adaptive threshold** — when the site **exports** ($P < 0$), the AC leakage limit is tightened **30 → 27 mA** via a debounced, transition-only FC16 write, and restored to 30 mA on return to import.

Both paths reuse the confirmed register map (S1), the live read (S2), the FC16 write-with-read-back (S3), and feed the reverse-flow flag into the S5 nomination confidence de-rate.

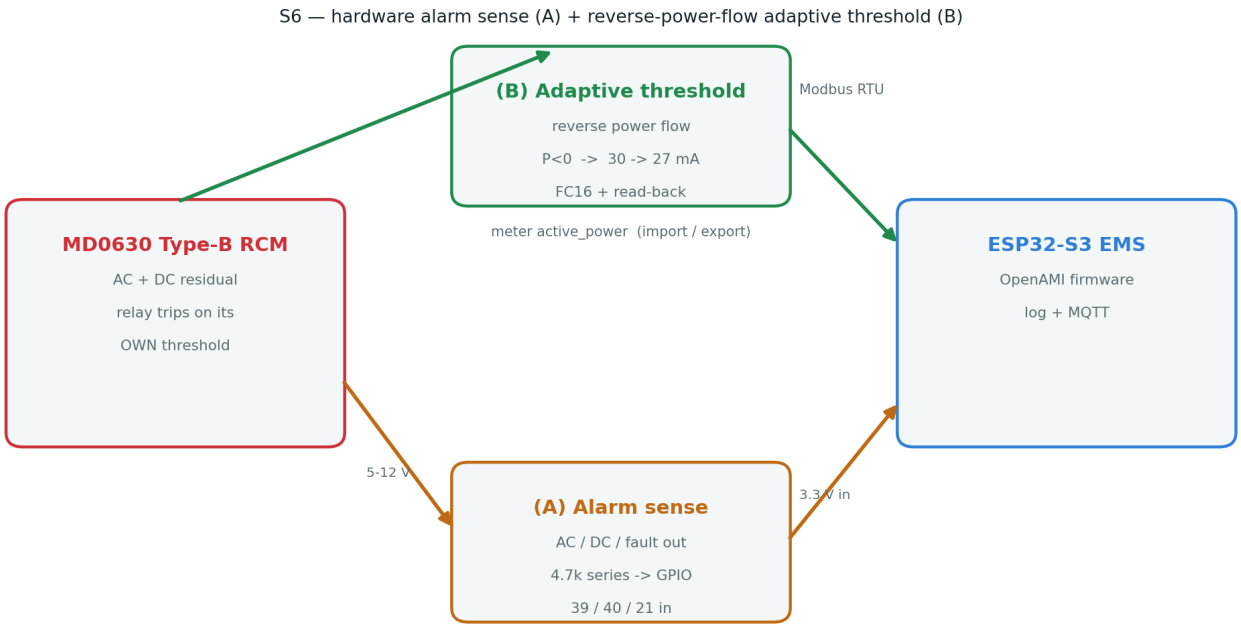

Figure 1: S6 architecture — alarm sense (A) and reverse-flow adaptive threshold (B) between the MD0630 and the ESP32.

2 (B) Reverse-power-flow adaptive threshold

Rationale. Under export (PV / battery back-feed) the residual-current signature and the acceptable leakage budget change; a slightly tighter AC limit (27 mA) protects the export path without nuisance-tripping normal import operation.

Detection. Export is read from the meter's active power: `readings[0].active_power < -0.05f` ($<$ -50 W). A build flag `-DLEAKAGE_REVERSE_FLOW_DEMO` substitutes a 20 s toggle so the write path can be exercised with no back-feed on the bench.

Write discipline (protects the module's store). The threshold is written **only on a debounced transition** (3 consecutive poll cycles in the new state), never every cycle — same FC16 + read-back path proven in S3:

```
static void update_reverse_flow_threshold(bool exporting_now, uint32_t now) {
    if (exporting_now == rf_exporting) { rf_streak = 0; return; }
    if (++rf_streak < 3) return;                  // debounce 3 poll cycles
    rf_streak = 0;
    rf_exporting = exporting_now;
    float target = rf_exporting ? Modbus_MD0630::AC_THRESHOLD_EXPORT_MA   // 27 mA
                                : Modbus_MD0630::AC_THRESHOLD_MA;         // 30 mA
    md0630.writeThreshold_mA(Modbus_MD0630::CH_AC, target);               // FC16 + read-back
}
```

The live export flag is also passed into `leakageInsights5.updateSelf(..., exporting_now, now)` so S5 `nominate()` de-rates confidence under reverse flow (a leak looks different when exporting).

Expected serial on transition (demo flag, 20 s toggle):

```
S6: reverse power flow EXPORT -> writing AC threshold 27 mA
S6: reverse power flow import -> writing AC threshold 30 mA
```

3 (A) Hardware alarm sense (read-only)

`include/metering/leakage_alarm_sense.h` — a lightweight `LeakageAlarmSense` that samples the module's **AC / DC / fault** alarm outputs and reports **only on an edge**:

```
bool poll() {                       // true if any line changed since last poll
    bool changed = false;
    for (auto& l : lines_) {
        l.active = read_active(l.pin);              // active-low through opto
        if (l.active != l.last) { l.last = l.active; changed = true; }
    }
    return changed;
}
```

Wired in `poll_leakage()`:

```
if (leakageAlarms.poll())
    Serial.printf("S6 ALARM edge: AC=%d DC=%d FAULT=%d\n",
                  leakageAlarms.ac(), leakageAlarms.dc(), leakageAlarms.fault());
```

Wiring (safety, no multimeter needed). The module's alarm outputs are 5–12 V; ESP32 GPIO are **not** 5 V-tolerant, so each output goes through **one 4.7 kΩ resistor in series** into an `INPUT_PULLUP` GPIO, read **active-low**. A series resistor is safe for *any* output type: an open-collector reads LOW when active, and a push-pull 5 V high is clamped by the GPIO's internal ESD diode with the resistor limiting the injected current to ~0.36 mA. Use the module's **5 V header** outputs, share **GND**, and rely on the **internal** pull-up (no 3V3/5V rail wired). Pins on the bare **ESP32-S3-DevKitC-1**: **GPIO 39 (AO/AC), 40 (DO/DC), 21 (AD/fault)** — free, non-strapping header pins (GPIO 33/34 are not broken out on the devkit; remap on a custom PCB). This path is **sense-only**: it observes the module's own trip, it does not command it.

4 Validation

Table 1: Firmware builds clean with S6 disabled (no regression) and with every S6 path compiled in.

Build configuration	Result
Default (both S6 flags off)	**SUCCESS** — RAM 18.8 %, Flash 27.6 %
`-DLEAKAGE_ALARM_SENSE` `-DLEAKAGE_REVERSE_FLOW_DEMO` `-DLEAKAGE_S5_DEMO`	**SUCCESS** — RAM 18.8 %, Flash 27.9 %

Bench runsheet (module on the ESP32).

1. Flash with `-DLEAKAGE_REVERSE_FLOW_DEMO` (leave `LEAKAGE_MOCK` off, module connected) → watch the AC threshold switch **30 ⇄ 27 mA** every ~20 s in the serial log and confirm via a CT.exe read-back.
2. For S6-A: wire the opto board, flash with `-DLEAKAGE_ALARM_SENSE`, induce a leak past 30 mA → confirm the `S6 ALARM edge: AC=1 …` line appears as the module relay trips.

5 Status & next

- **S6 complete (firmware)** — reverse-flow adaptive threshold (B) and hardware alarm sense (A) implemented, gated behind build flags, and compiling in both the default and full-S6 configurations. S6-B reuses the S3 write path; S6-A adds a read-only observer.
- **Bench capture** — run the runsheet above on the module to attach the serial screenshots (same as S3 read-back / S5 charts).
- Production polish: publish the alarm-line state and the active AC threshold on the MQTT feed alongside `rcm_ac_mA` / `rcm_dc_mA`.

Repository: `docs/leakage_MD0630TA1A/` — completes the `S1`–`S5` set. Together S1–S6 form the full technical document: register map → live read → threshold write → energy correlation → nomination → hardware alarm + adaptive threshold.